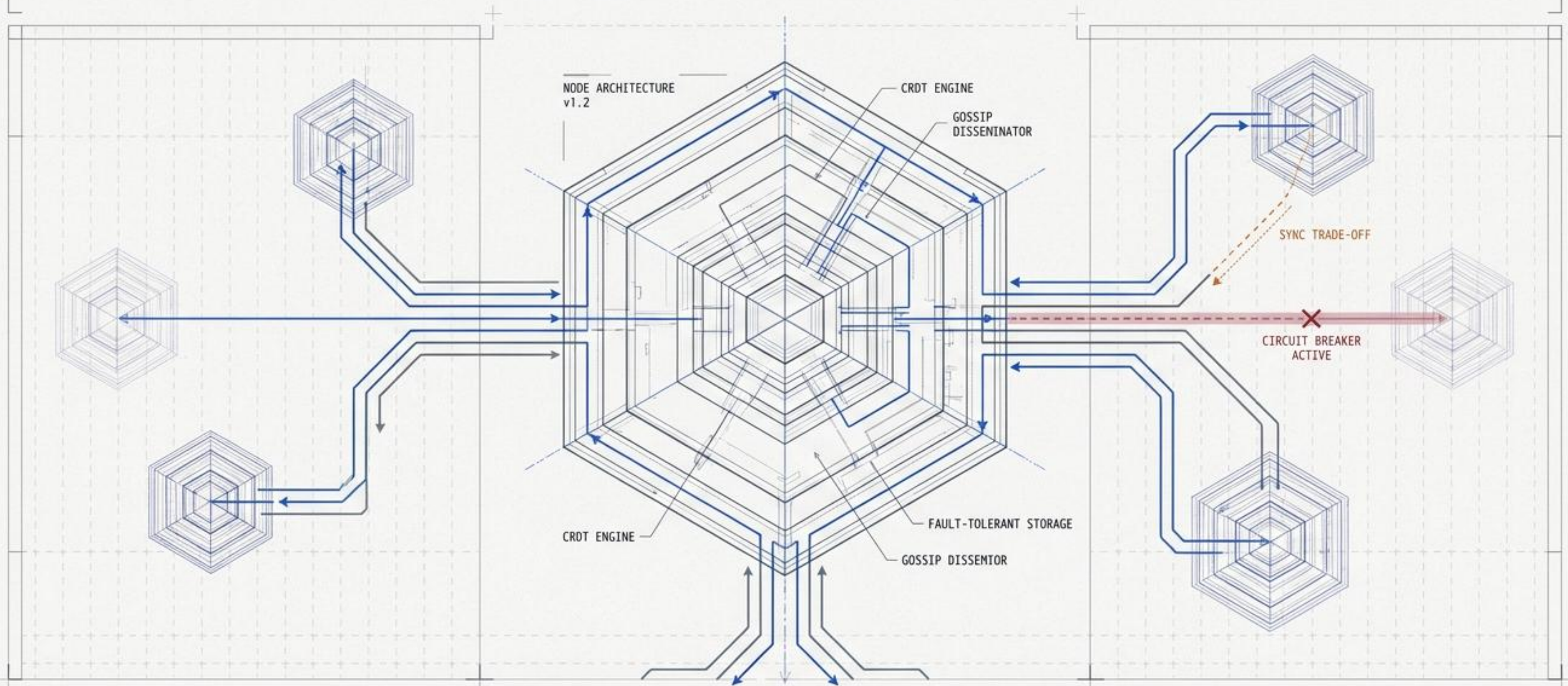


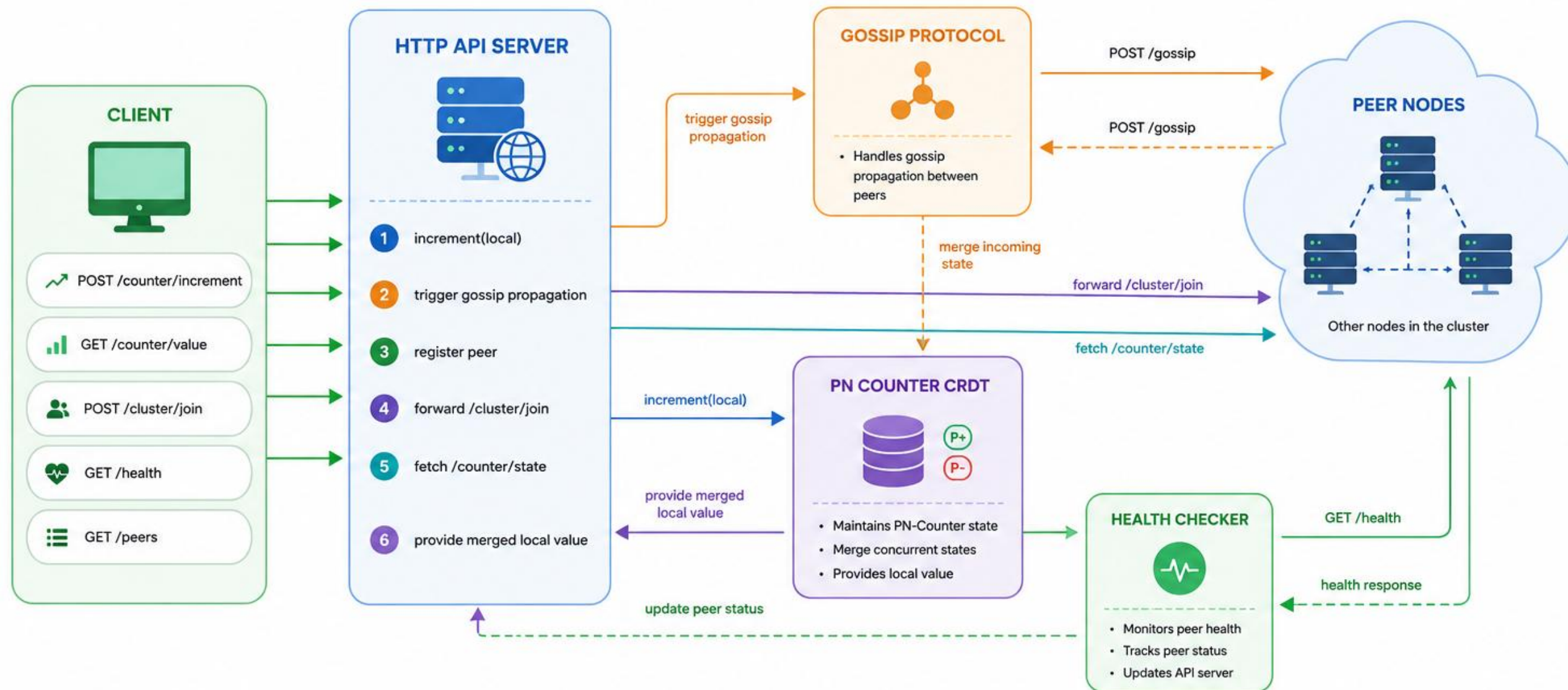
BLUEPRINT ARSITEKTUR: SISTEM DISTRIBUTED COUNTER

Membangun sistem tanpa Single Point of Failure menggunakan Go, CRDT, dan Gossip Protocol.



Distributed Counter System Architecture

High-level overview of components and interactions



FLOW LEGEND

- Local counter operations
- Gossip propagation
- Health monitoring
- Cluster management
- State synchronization
- Response / Return flow

COMPONENT OVERVIEW



HTTP API SERVER

Exposes REST API and orchestrates operations across components



GOSSIP PROTOCOL

Disseminates state updates among peer nodes



PN COUNTER CRDT

Conflict-free replicated data type for the distributed counter



HEALTH CHECKER

Monitors health of peers and maintains cluster status



PEER NODES

Other instances in the cluster participating in gossip and state sync

KEY BENEFITS

- Scalable & Decentralized
- Fault Tolerant
- Eventually Consistent
- Highly Available

Kerapuhan Sistem Terpusat

Masalah Utama: Bagaimana menjaga nilai counter global tetap akurat di seluruh cluster tanpa memiliki satu titik pusat kegagalan?

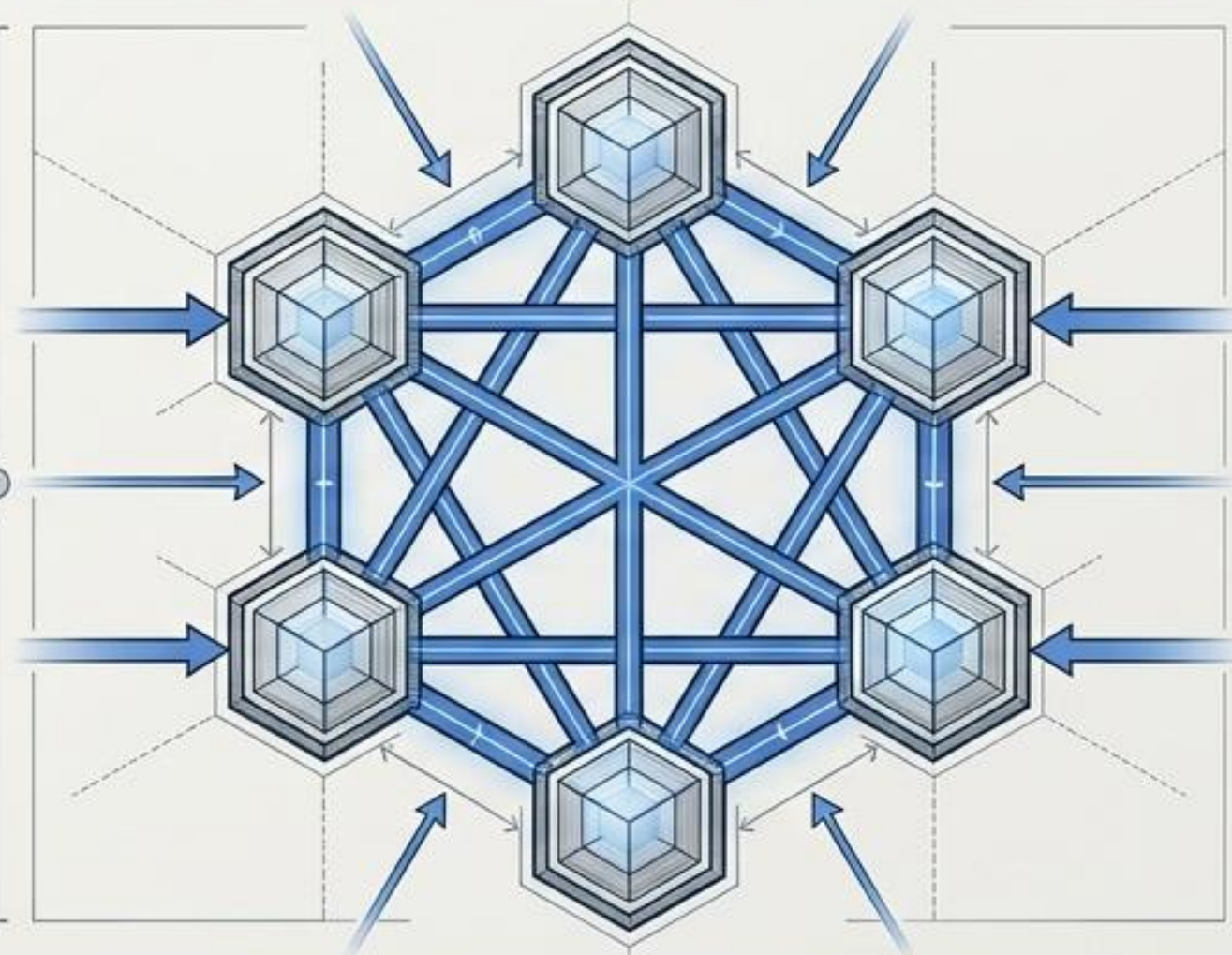
Ancaman: Update yang terjadi secara konkuren menyebabkan race conditions, dan matinya satu node utama dapat melumpuhkan seluruh sistem.

THE THREAT: **CENTRALIZED FRAGILITY**



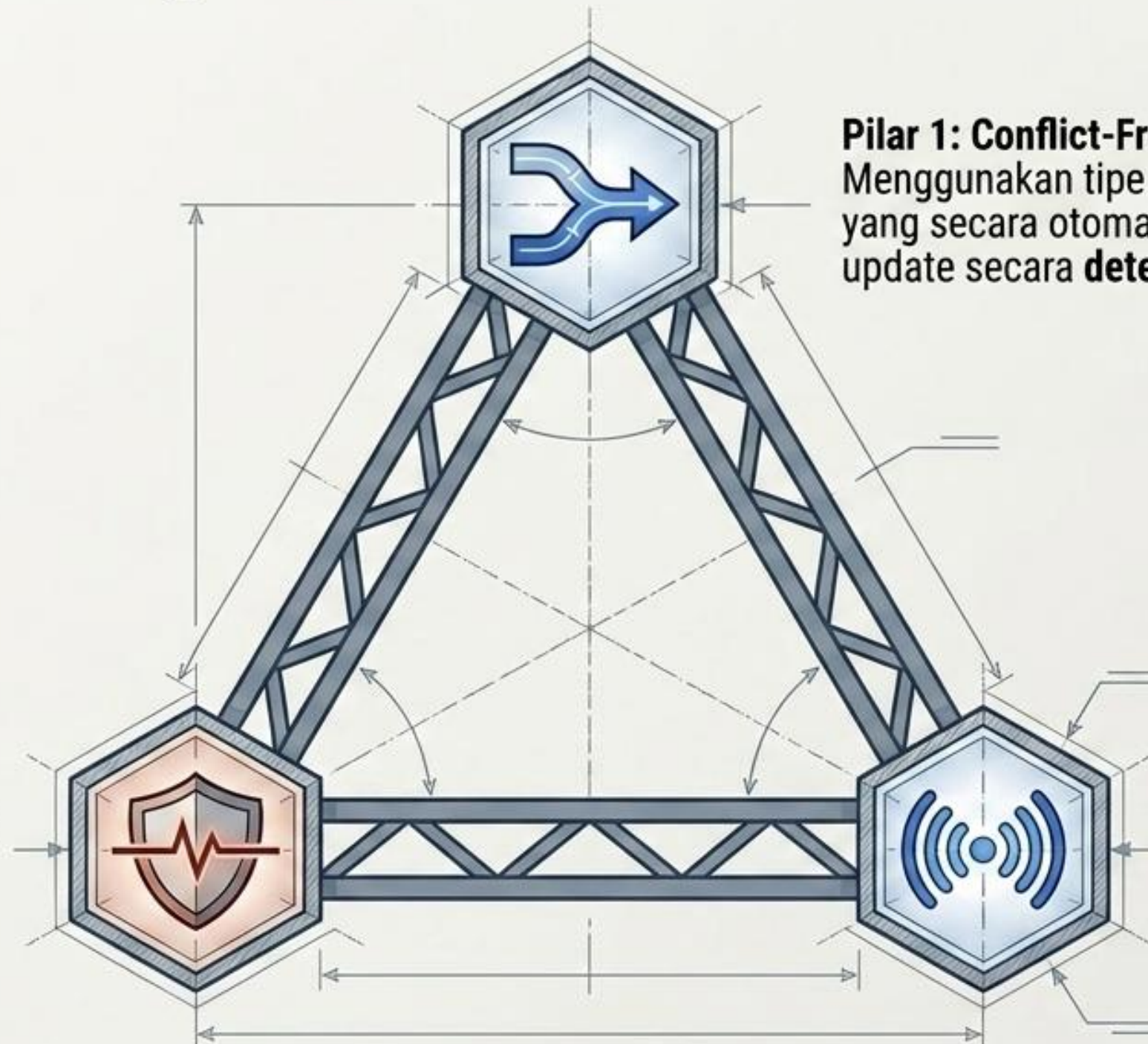
Kerapuhan Sistem Terpusat: Beban konkuren yang tinggi menyebabkan kegagalan katastrofik. Satu node basis data yang mati menghentikan seluruh operasi. **SINGLE POINT OF FAILURE.**

THE GOAL: **DECENTRALIZED RESILIENCE**



Tujuan: Mesh Node Terdistribusi. Beban dibagi secara merata, tanpa titik pusat kegagalan. Ketsahanan terhadap matinya node individu melalui **LOAD BALANCE, REDUNDANCY.**

Tiga Pilar Ketahanan Sistem

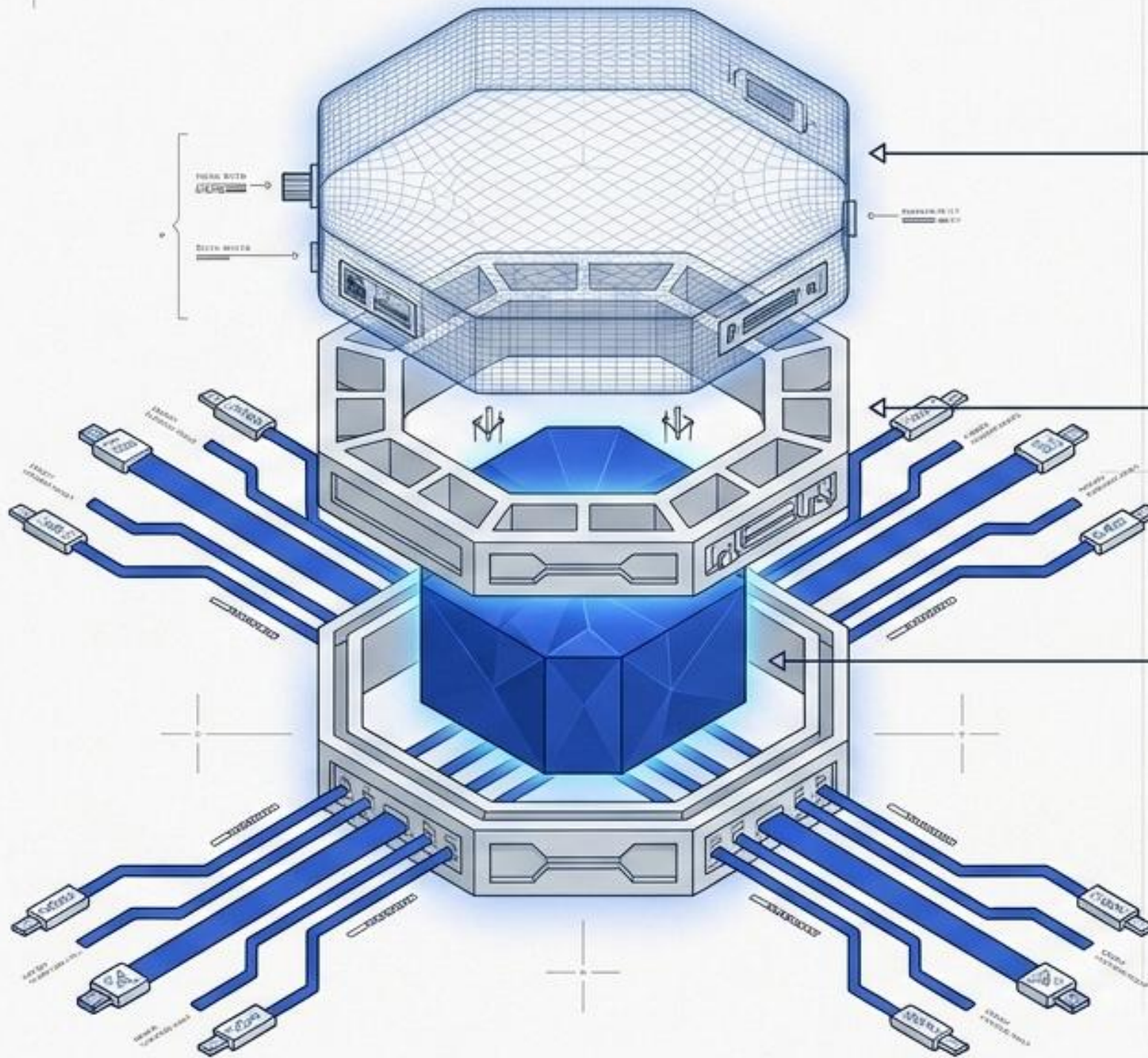


Pilar 1: Conflict-Free Merge (CRDT). Menggunakan tipe data khusus (**PN Counter**) yang secara otomatis menyelesaikan konflik update secara **deterministik**.

Pilar 3: Isolasi Kegagalan (Health Check). Mendeteksi *node* yang tidak responsif dan memutus jalurnya agar tidak mengganggu kinerja cluster.

Pilar 2: Sinkronisasi Tanpa Leader (Gossip). Menyebarkan *state* antar *node* secara eventual tanpa memerlukan koordinasi terpusat.

Anatomi Internal Sebuah Node



Layer 1: Antarmuka (HTTP API).

Menangani request dari client (`POST /counter/increment`, `GET /counter/value`).

Layer 2: Orkestrasi (Node Management).

Menyimpan daftar peer, status koneksi, dan jejak waktu (timestamp) interaksi terakhir.

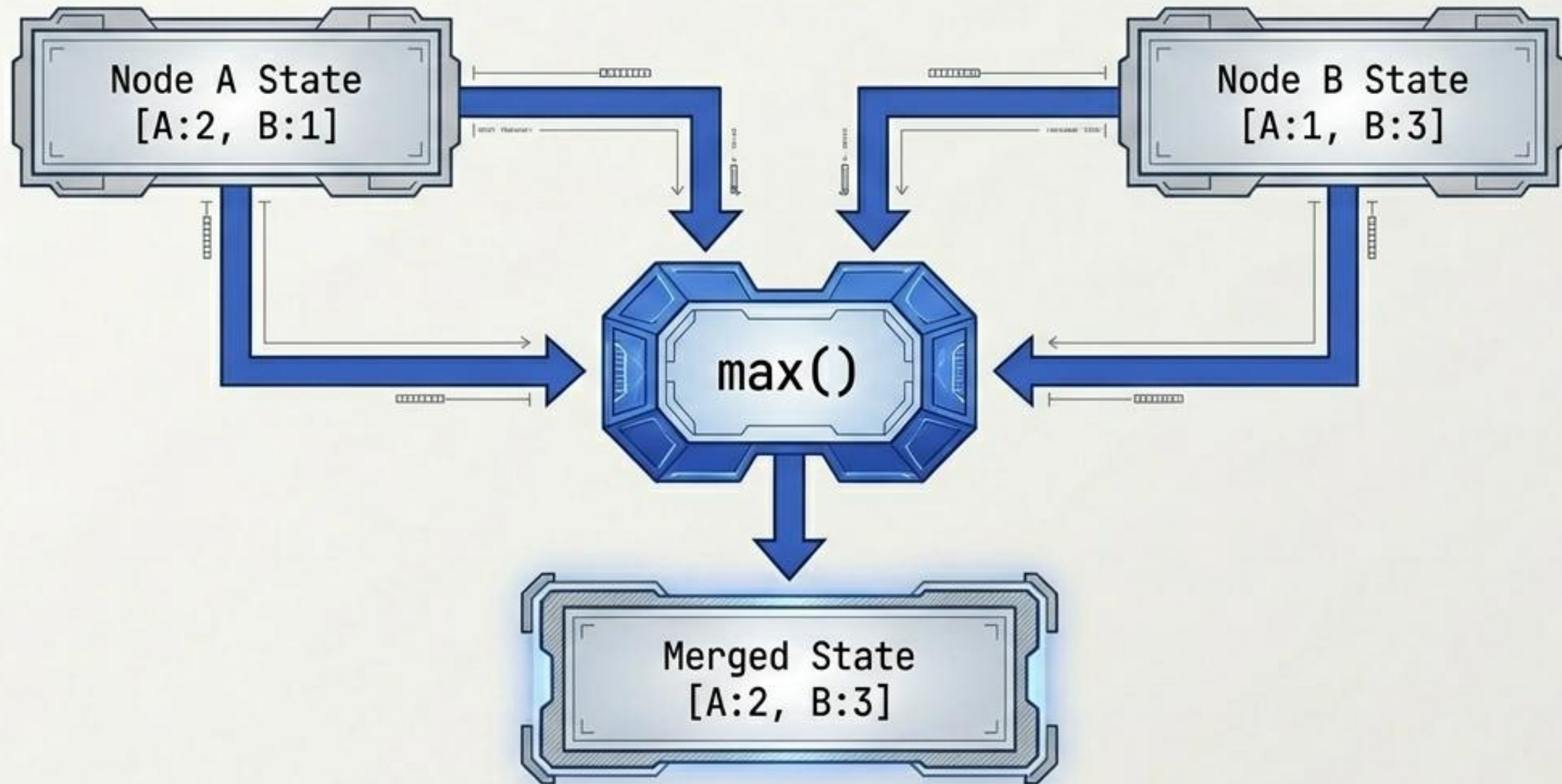
Layer 3: Inti Data (CRDT).

Menyimpan histori increment dan decrement absolut milik node ini.

Sistem Saraf (Gossip & Health).

Berkomunikasi dengan dunia luar melalui `GET /health` dan `POST /gossip`.

Resolusi Konflik Deterministik

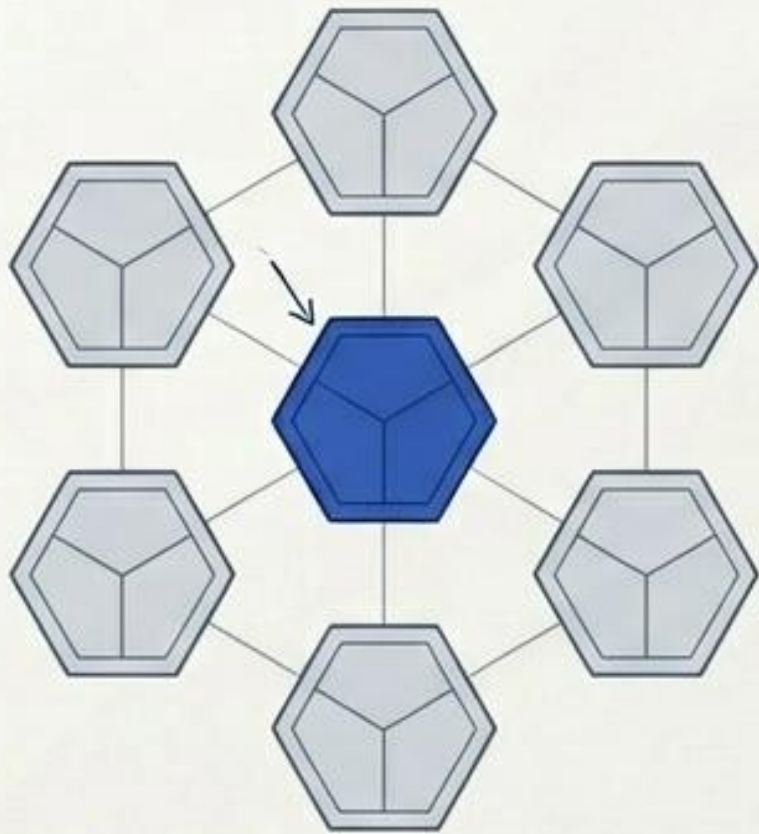


PN Counter CRDT menghilangkan kebutuhan akan **consensus** global. Setiap node berhak melakukan increment secara lokal. Saat state bertemu, sistem cukup mengambil nilai maksimum (max) dari setiap entri node untuk mencapai kebenaran absolut.

Propagasi Epidemik (Gossip Protocol)

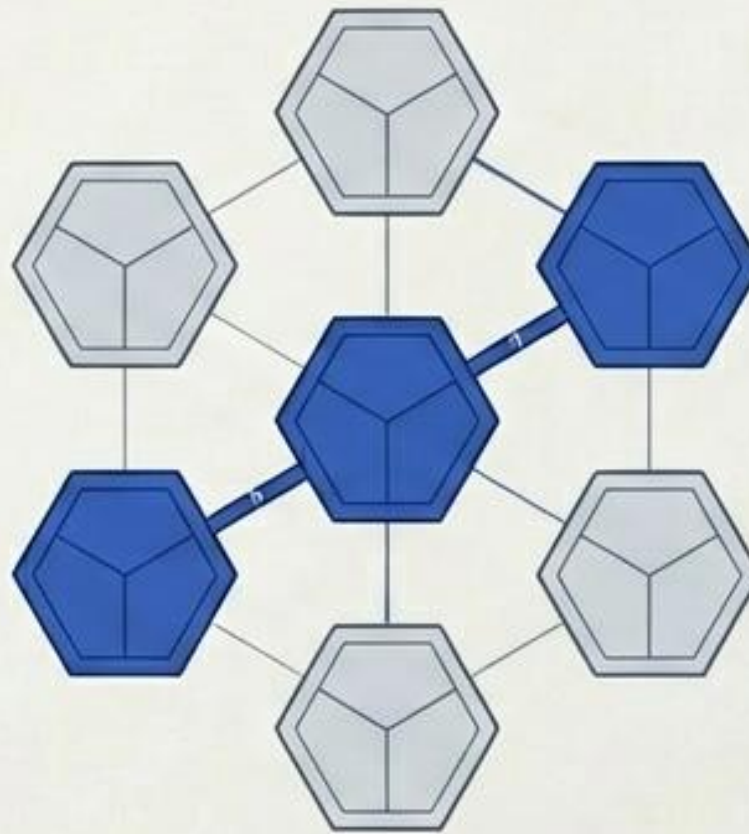
Eventual Consistency dicapai dengan menularkan state terbaru ke peer secara periodik.

Panel 1: Local Update.



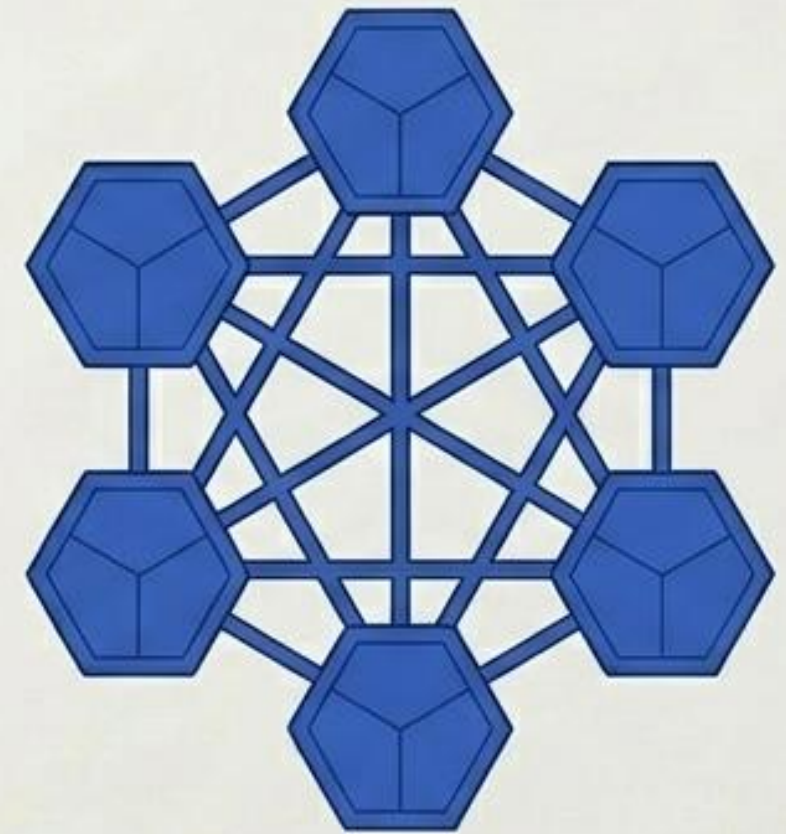
Node menerima instruksi mutasi data.

Panel 2: Gossip Tick.



State dikirimkan secara acak ke peer yang aktif.

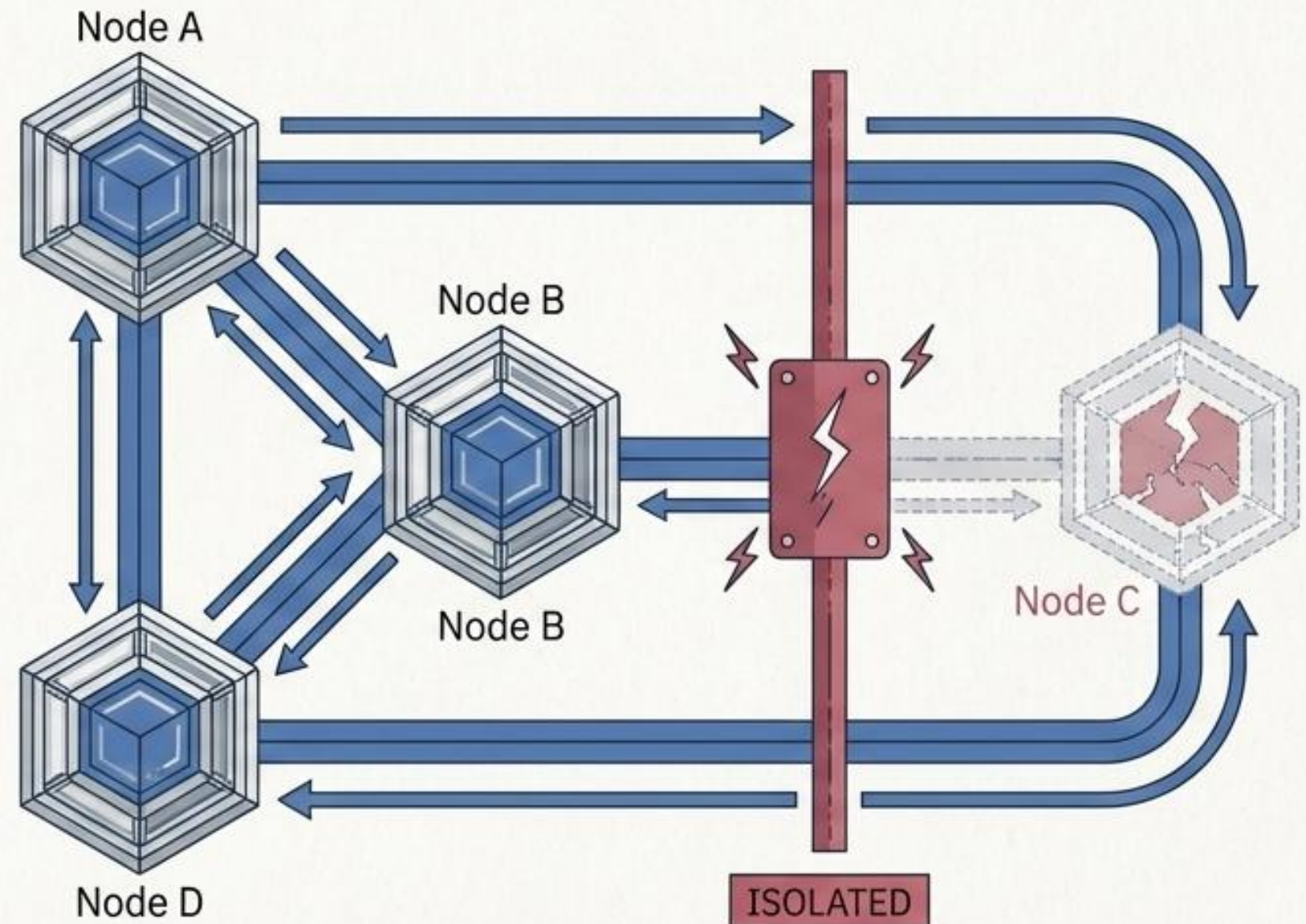
Panel 3: Convergence.



Seluruh cluster mencapai state yang sama secara eksponensial.

Katup Isolasi Kegagalan (Circuit Breaker)

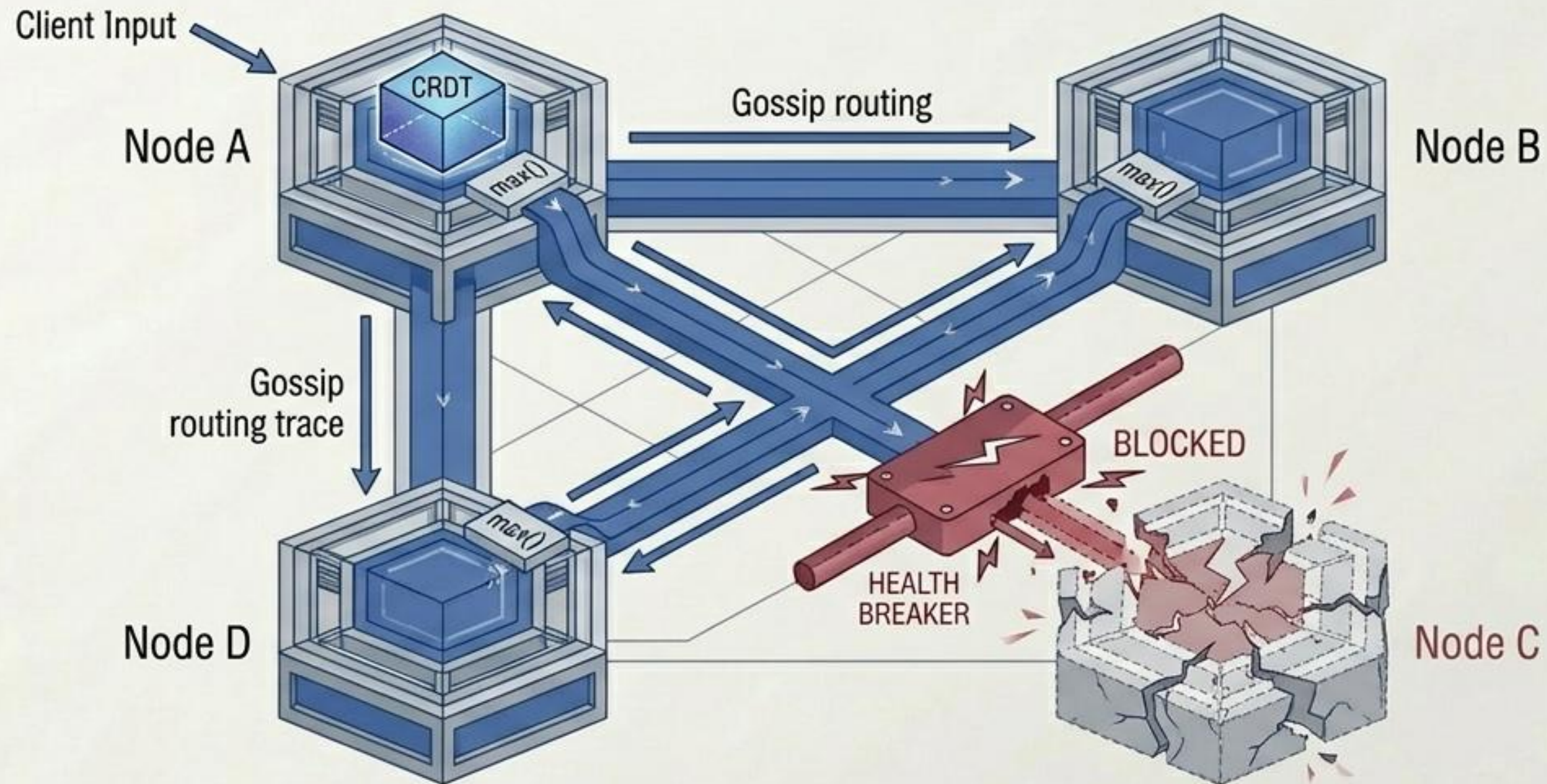
Modul health/checker.go terus melakukan ping ke peer via GET /health. Jika terjadi beberapa kali timeout, peer ditandai sebagai gagal (failed). Deteksi kegagalan ini mencegah Gossip Protocol membuang resource komputasi dan jaringan untuk menghubungi node yang sudah mati.



Katup isolasi kegagalan secara fisik memutus jalur komunikasi ke node yang mati, mencegah penyebaran data yang tidak perlu.

Harmoni Sistem Terdistribusi

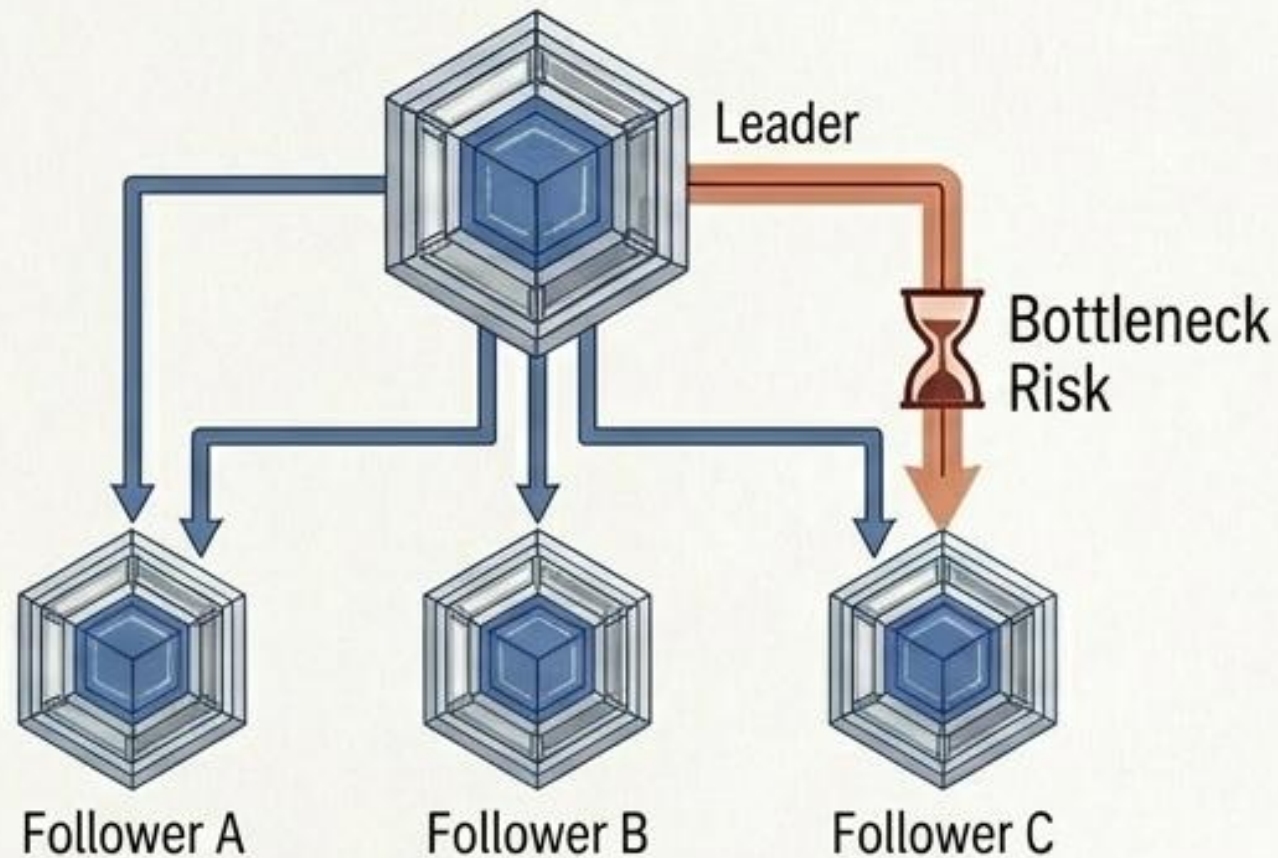
Sistem ini bukanlah sekumpulan komponen terpisah, melainkan sebuah organisme tunggal yang tangguh. CRDT mengelola kebenaran data, Gossip mendistribusikannya, dan Health Checker menjaga jalur distribusi tetap efisien dengan memotong rute ke node yang mati.



Keputusan Teknis: Menghindari Botol Leher Konsensus

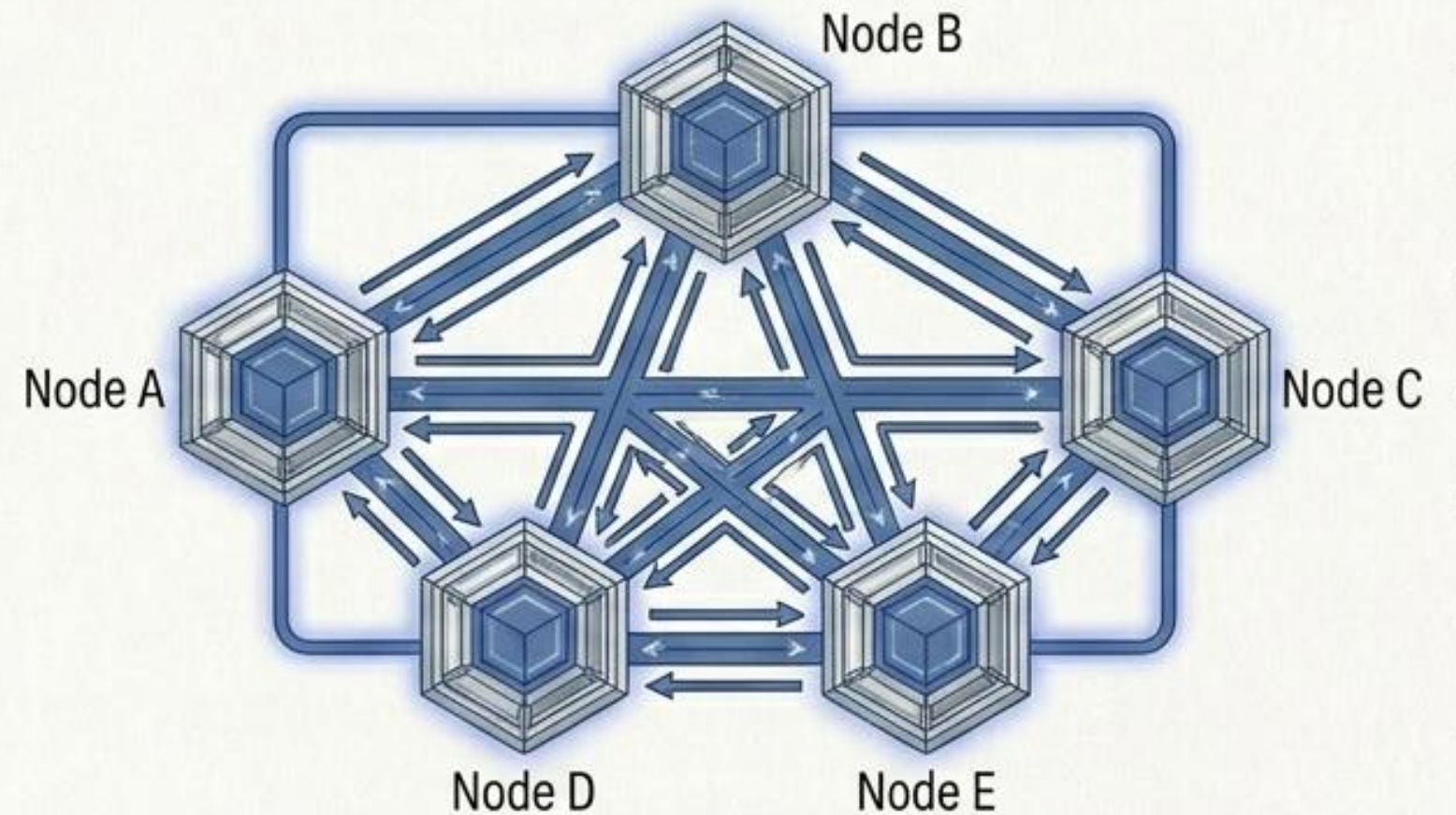


Strong Consensus (**Raft/Paxos**)



- Terdapat faksi Leader & Follower.
- Menjamin Strict Consistency.
- Rentan menjadi bottleneck jika Leader tumbang.

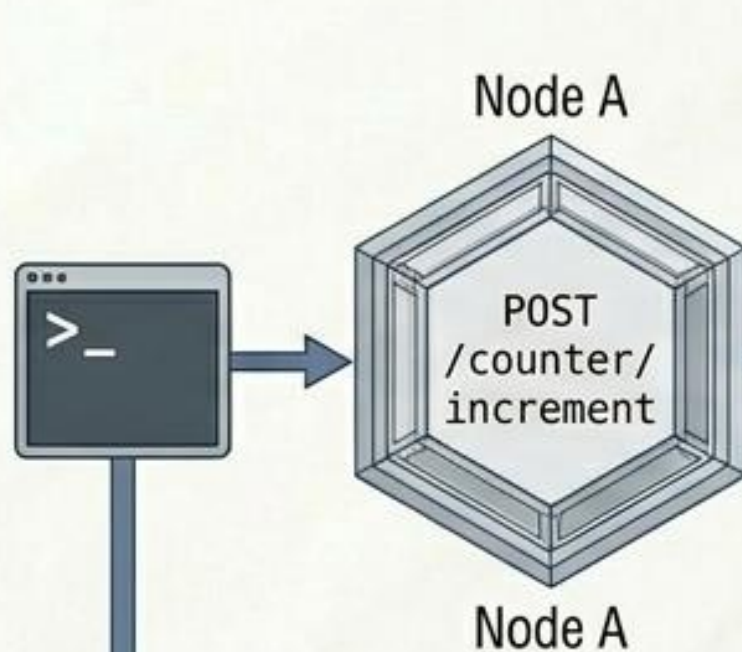
Leaderless + CRDT (**Pilihan Kita**)



- Semua node berstatus sama (Peer-to-Peer).
- Fokus pada Eventual Consistency.
- Tidak ada Single Point of Failure—setiap node melayani request secara mandiri.

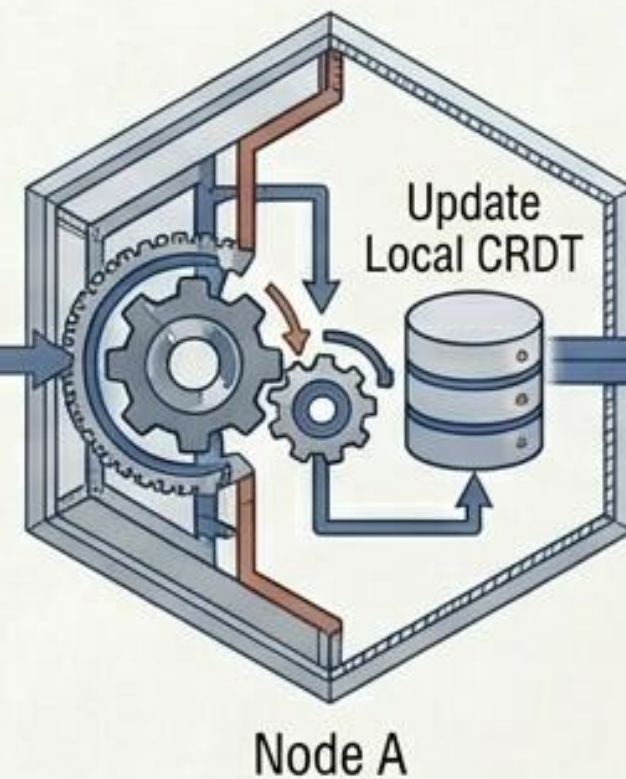
Alur Fisiologi: Mutasi Data (Increment)

Step 1: Injeksi



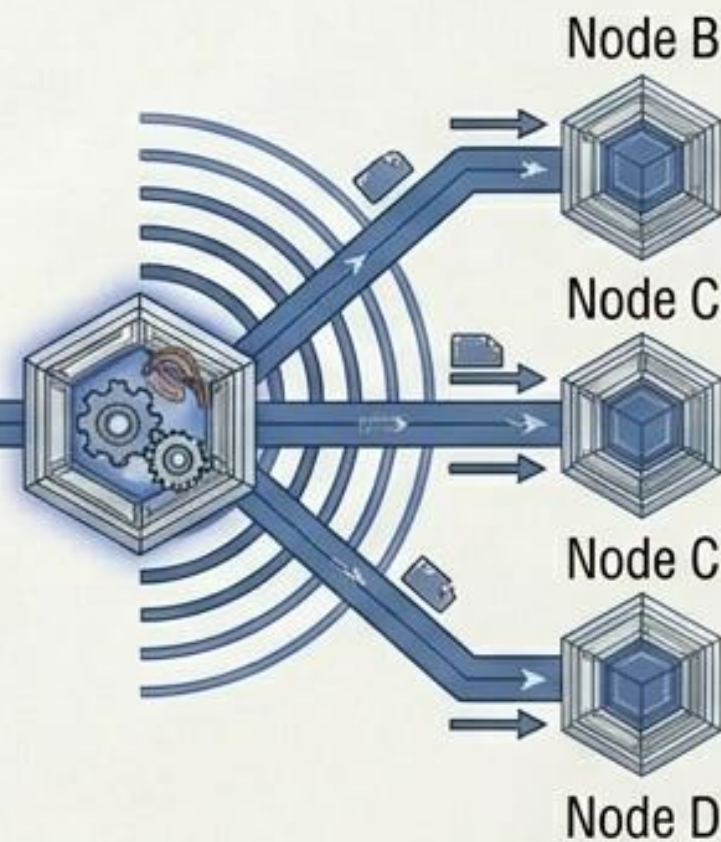
Klien melakukan POST ke sembarang node.

Step 2: Mutasi Lokal



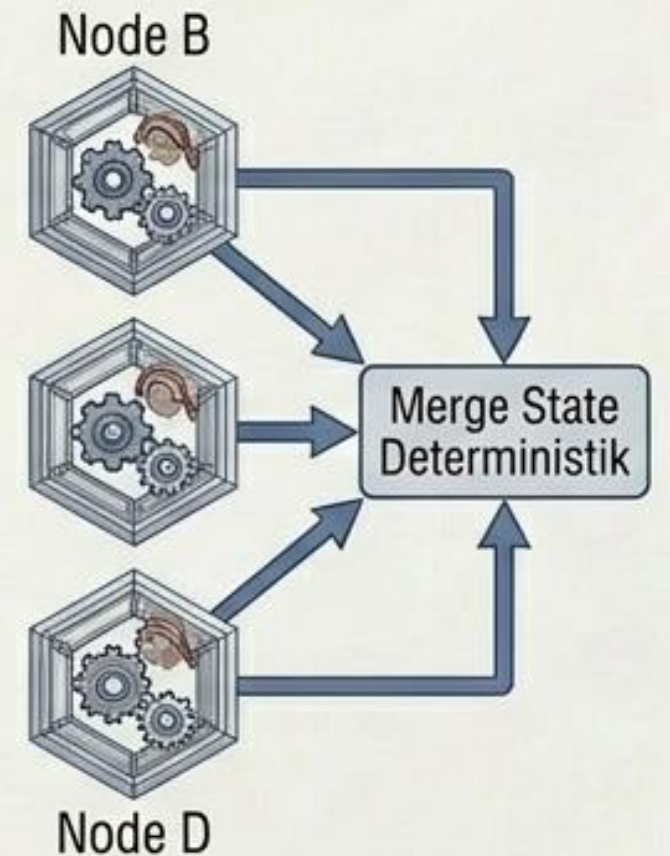
Node segera memperbarui CRDT miliknya.

Step 3: Propagasi



Node memicu gossip ke seluruh peer yang aktif.

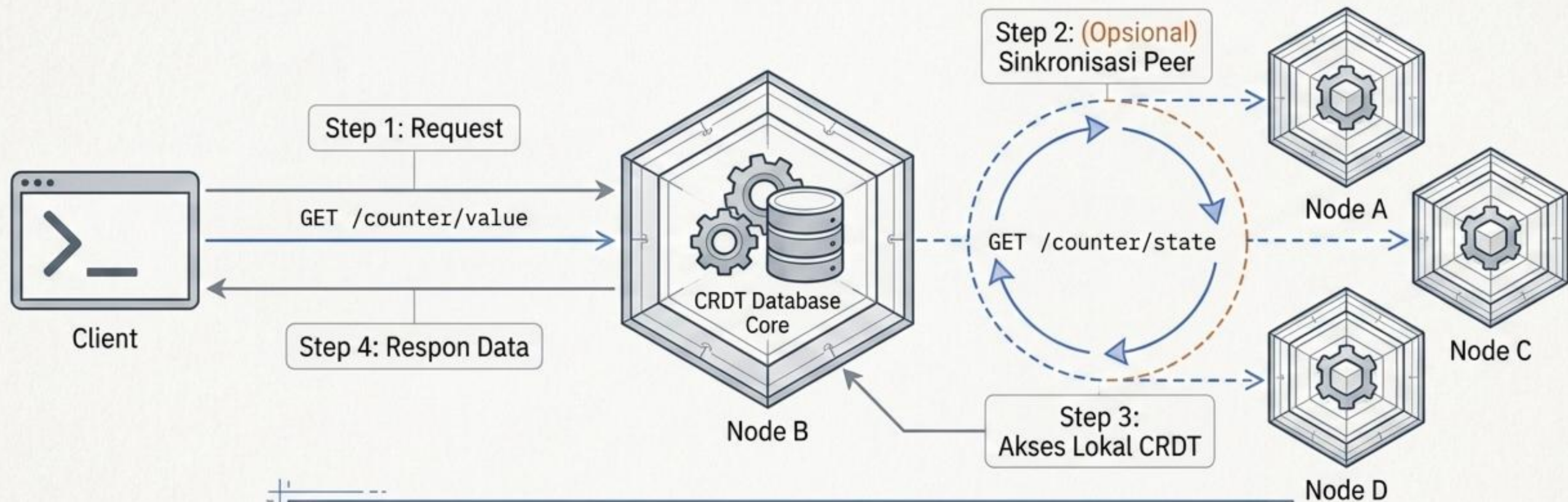
Step 4: Konvergensi



Node lain menerima paket data dan melakukan merge state secara asinkron.

Alur Fisiologi: Ekstraksi Data (Read)

Node pada dasarnya mengembalikan nilai kalkulasi langsung dari state CRDT lokalnya.

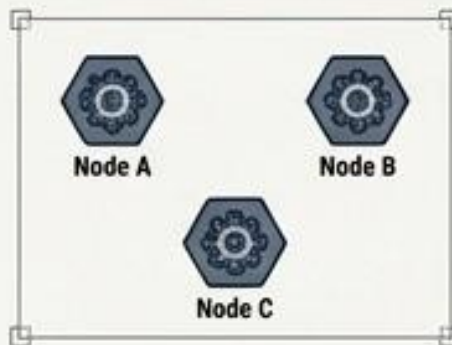


Pro-Tip Mitigasi: Sebelum memberikan respon, node dapat secara proaktif menarik state terbaru dari peer aktif untuk meminimalisir perbedaan data sementara (stale read) antar node dalam cluster.

Rencana Simulasi Cluster (Penerbangan Uji)

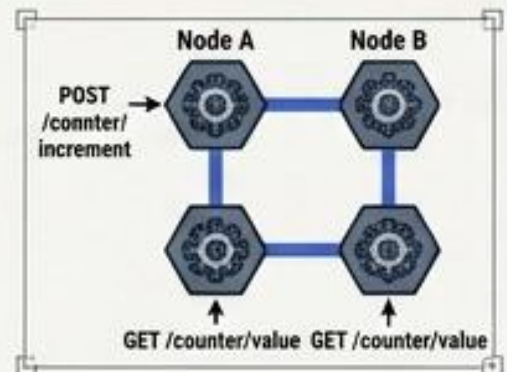
Fase 1: Inisialisasi

Jalankan 3 node terpisah.
Verifikasi dengan **GET /peers**
dan **GET /health**.



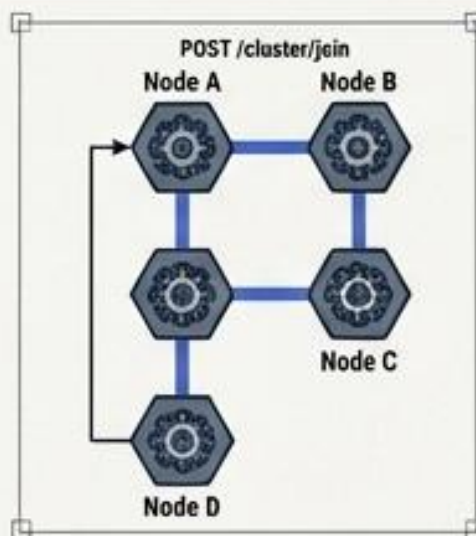
Fase 2: Mutasi & Verifikasi

Eksekusi **POST /counter/increment** pada Node A.
Pantau propagasi dengan **GET /counter/value** pada Node B dan C.



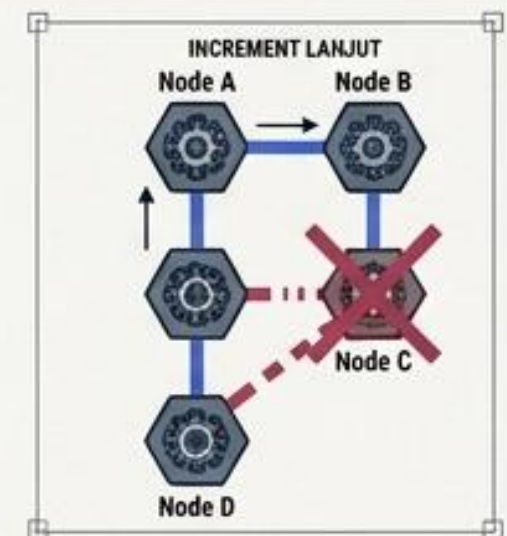
Fase 3: Registrasi Dinamis

Injeksi node ke-4 secara runtime menggunakan **POST /cluster/join**.



Fase 4: Uji Ketahanan

Matikan 1 node secara paksa dan demonstrasikan increment tetap berjalan mulus di sisa cluster.



Neraca Diagnostik Arsitektur

Kekuatan (Robustness)



Sangat toleran terhadap kegagalan;
desain leaderless sejati.



CRDT menyelesaikan bentrok
mutasi tanpa intervensi.



Gossip & Health check menjaga
ketersediaan secara otonom.

Keterbatasan (Trade-offs)



Data rentan hilang saat restart
karena ketiadaan disk persistence.



Konsistensi data bersifat eventual;
akan ada delay keseragaman
bacaan sementara.



Mekanisme propagasi registrasi
node (/join) belum didesain untuk
cluster berskala masif.

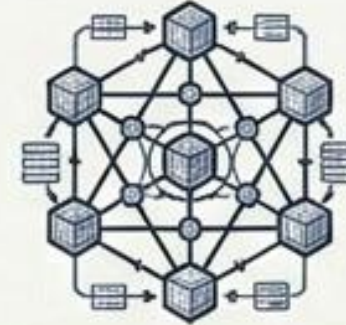
Peta Jalan Evolusi Sistem



Level 1:
Durabilitas Data.
Menambahkan penyimpanan lokal/
on-disk.



Level 2: Strict Quorum. Opsi implementasi quorum read/write bagi client yang membutuhkan konsistensi absolut (*strong consistency*).



Level 3: Protokol Keanggotaan.
Mengganti sistem join sederhana dengan protokol yang tangguh untuk skala besar (seperti Gossip Membership atau SWIM).



Level 4: Observabilitas.
Membangun dashboard monitoring komprehensif untuk cluster.

REFERENSI ANALITIS: Evaluasi Kritis Terhadap Pola Desain Terdistribusi.

Blueprint Ketahanan Terbukti

Sistem ini membuktikan bahwa distributed counter yang absolut dapat dibangun dengan elegan. Tanpa figur leader, tanpa titik rawan tunggal.

Takeaways Akhir:

- Setiap node memiliki otoritas melayani client.
- Bentrok data tunduk pada matematika deterministik CRDT.
- Eventual consistency bukan kelemahan, melainkan harga rasional untuk ketersediaan sistem tanpa henti.

